

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from google.colab import files
import io
```

```
print("Please upload an image file (e.g., .jpg, .png, .bmp, .tif).")

uploaded = files.upload()

if not uploaded:
    print("No file uploaded.")
else:
    image_filename = next(iter(uploaded))

    # Convert uploaded bytes into image
    image = cv2.imdecode(
        np.frombuffer(
            uploaded[image_filename],
            np.uint8
        ),
        cv2.IMREAD_COLOR
    )

    if image is None:
        print("Error: Unable to load the selected image.")
    else:
        print("Image loaded successfully.")
        print("Selected Image:", image_filename)
```

Please upload an image file (e.g., .jpg, .png, .bmp, .tif).

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this

cell to enable.

Saving leap.jpg to leap.jpg

Image loaded successfully.

```
# STEP 3: CONVERT IMAGE TO GRAYSCALE
# AND APPLY GAUSSIAN BLUR

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

blurred = cv2.GaussianBlur(gray, (5, 5), 0)

# Display grayscale and blurred image

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(blurred, cmap="gray")
plt.title("Gaussian Blurred Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Grayscale Image



Gaussian Blurred Image



```
_, global_thresh = cv2.threshold(  
    blurred,  
    127,  
    255,  
    cv2.THRESH_BINARY  
)  
  
plt.figure(figsize=(6, 4))  
plt.imshow(global_thresh, cmap="gray")  
plt.title("Global Thresholding")  
plt.axis("off")  
plt.show()
```

Global Thresholding



```
otsu_value, otsu_thresh = cv2.threshold(  
    blurred,  
    0,  
    255,  
    cv2.THRESH_BINARY + cv2.THRESH_OTSU  
)  
  
print("Otsu Threshold Value:", otsu_value)  
  
plt.figure(figsize=(6, 4))  
plt.imshow(otsu_thresh, cmap="gray")  
plt.title("Otsu Thresholding")  
plt.axis("off")  
plt.show()
```

Otsu Threshold Value: 133.0

Otsu Thresholding



```
binary = otsu_thresh.copy()

kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(
    binary,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
)

sure_bg = cv2.dilate(opening, kernel, iterations=3)

dist_transform = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)

_, sure_fg = cv2.threshold(
    dist_transform,
    0.5 * dist_transform.max(),
    255,
    0
)

sure_fg = np.uint8(sure_fg)

unknown = cv2.subtract(
    sure_bg,
    sure_fg
)

_, markers = cv2.connectedComponents(sure_fg)

markers = markers + 1

markers[unknown == 255] = 0

watershed_image = image.copy()

markers = cv2.watershed(
    watershed_image,
    markers
)

watershed_image[markers == -1] = [255, 0, 0]

plt.figure(figsize=(7, 5))
plt.imshow(
    cv2.cvtColor(watershed_image, cv2.COLOR_BGR2RGB)
)
plt.title("Watershed Segmentation")
plt.axis("off")
plt.show()
```

Watershed Segmentation



```
plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(global_thresh, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu Thresholding")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(cv2.cvtColor(watershed_image, cv2.COLOR_BGR2RGB))
plt.title("Watershed")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Global Thresholding



Otsu Thresholding



Watershed

